

BITÁCORA COMPLETA DE DESARROLLO ASISTIDO POR IA

Registro 1:

- **Prompt utilizado:** “Respecto a los requerimientos que me piden para crear este proyecto me puedes dar el código utilizando las estructuras o los elementos que se piden para su funcionamiento correcto, intenta seguir un desarrollo simple y fácil de entender al momento de exponerlo oralmente, no utilices cosas complejas y mantén un orden claro”
- **Respuestas generadas:** “Para que sea fácil de exponer oralmente y no te metas en complicaciones con la integración de lenguajes (como usar llamadas nativas con ctypes o Cython), lo mejor y más limpio es usar un enfoque basado en archivos de texto (CSV): Python genera los procesos en un archivo, tu programa en C los lee, simula todo, exporta los resultados a otro archivo, y Python los grafica.” “Dado que el proyecto es extenso, vamos a estructurar la base sólida del código en **C** con los elementos obligatorios que te piden: las estructuras del proceso y de memoria (usando **renglones** para cualquier lógica de bloques), el Scheduler FIFO con una Cola, y la Lista Doblemente Ligada para la memoria.” “El Archivo de Cabecera: simulador.h”, “Implementación Principal: simulador.c”, “El Script de Python para cumplir la Integración: analisis.py”
- **Errores encontrados:** Por el momento ninguno ya que modificamos el prompt
- **Correcciones realizadas:** Corregimos el prompt para una mejor adaptación a lo solicitado
- **Validaciones realizadas:** Ninguna por el momento

Registro 2:

- **Prompt utilizado:** “Todo lo que antes ya me habías dado ahora lo puedes implementar en estos archivo, modificarlos para que tenga un funcionamiento correcto, fácil de entender y de explicar ”, Se añadieron los archivos *circular_queue.c*, *coalescence.c*, *fifo.c*, *first_fit.c*, *queue.c*, *round_robin.c*, *sjf.c*, *stack.c*
- **Respuestas generadas:** “Para que tu proyecto tenga un funcionamiento correcto, esté completamente alineado con la estructura modular que subiste y sea sumamente fácil de explicar ante los profesores, vamos a reescribir y adaptar el contenido de tus archivos. He unificado la lógica utilizando nombres claros en español (como **renglones** para los bloques de memoria) y agregando comentarios detallados línea por línea que te servirán como guión para tu exposición oral. Aquí tienes la implementación ordenada y lista para cada archivo:”, Comenzamos con la estructura del proyecto en los archivos *proceso.h* , *memory_manager.h* , *first_fit.c* , *coalescence.c* , *queue.c* , *circular_queue.c* , *stack.c* , *scheduler.h* , *fifo.c* , *round_robin.c* , *sjf.c* , *main.c*”
- **Errores encontrados:** Indentación en los códigos, los escribió de tal manera que no se notaba las estructuras y olvido generar los archivos *main.c* y *makefile*, además los archivos *.h* que la misma IA colocó en los *.c*
- **Correcciones realizadas:** Se corrigieron las indentaciones manualmente, para que quedara lo mejor posible y posteriormente se solicitaron los archivos faltantes
- **Validaciones realizadas:** Se validó que los archivos estuvieran identados correctamente

Registro 3:

- **Prompt utilizado:** “Ahora dame el ejecutable”
- **Respuestas generadas:** “Para poder compilar y generar el archivo ejecutable de tu simulador de sistema operativo, necesitamos un archivo main.c que integre de manera lógica todos los módulos (procesos, memoria, colas, pila y planificadores) y simule un ciclo de ejecución real. Además, para automatizar la compilación de forma profesional, utilizaremos un archivo Makefile. A continuación tienes el código del punto de entrada del sistema (main.c) y las instrucciones exactas para compilar y obtener tu ejecutable.”
- **Errores encontrados:** Indentación en los códigos, los escribió de tal manera que no se notaba las estructuras y siguen faltando los archivos .h
- **Correcciones realizadas:** Se corrigieron las indentaciones manualmente, para que quedara lo mejor posible y se solicitaron posteriormente los archivos faltantes
- **Validaciones realizadas:** Se validó que los archivos estuvieran identados correctamente y que los archivos no tuvieran errores al compilar

Registro 4:

- **Prompt utilizado:** “dame todos los .h de los archivos”
- **Respuestas generadas:** “Para que tu proyecto esté 100% completo, bien estructurado y compile perfectamente de forma modular con el comando de gcc que usamos anteriormente, aquí tienes el código fuente de todos los archivos de cabecera (.h). Cada interfaz del Tipo Abstracto de Datos (TAD) han sido guardas de inclusión (#ifndef) para evitar duplicaciones en el preprocesador y comentarios detallados en cada renglón.” Se proporcionaron los archivos process.h , memory_manager.h , queue.h , circular_queue.h , stack.h , scheduler.h”
- **Errores encontrados:** Indentación en los códigos, los escribió de tal manera que no se notaba las estructuras
- **Correcciones realizadas:** Se corrigieron las indentaciones manualmente, para que quedara lo mejor posible y se detectó un error al momento de ejecutar el programa que despues se soluciono
- **Validaciones realizadas:** Se validó que los archivos no tuvieran errores al compilar

Registro 5:

- **Prompt utilizado:** “resuelve este error”

```
PS C:\Users\Joshua1\Documents\PROGRAMAS\ProyectoFinal_EDA> gcc -Wall -Wextra -std=c99 -g -o simulador_os
main.c first_fit.c coalescence.c queue.c circular_queue.c stack.c fifo.c round_robin.c sjf.c
sjf.c: In function 'sjf_next':
sjf.c:55:9: warning: unused variable 'pid' [-Wunused-variable]
  55 |     int pid = s->processes[0].pid; // Rescatamos el ID de la tarea más corta (índice cero)
      |         ^~~~~
PS C:\Users\Joshua1\Documents\PROGRAMAS\ProyectoFinal_EDA> |
```

- **Respuestas generadas:** “El error que te está apareciendo en la terminal se debe a una inconsistencia de firmas en las funciones de los planificadores (fifo.c, round_robin.c, sjf.c). En tu archivo scheduler.h, definiste que la función para obtener el siguiente proceso debe llamarse scheduler_next, sin embargo, en el archivo round_robin.c escribiste rr_next_tick y en sjf.c escribiste sjf_next. Además, en tu archivo main.c, intentaste simular el comportamiento de Round Robin de forma manual usando cq_dequeue directamente sobre la cola circular, en lugar de invocar de manera uniforme una interfaz común de planificación, lo que rompe con la modularidad que pide la profesora.”
- **Errores encontrados:** Error por usar el tipo de dato Scheduler en distintos archivos
- **Correcciones realizadas:** Se modificaron los archivos round_robin.c , sjf.c y se actualizó scheduler.h
- **Validaciones realizadas:** Se validó que los archivos no tuvieran errores al compilar

Registro 6:

- **Prompt utilizado:** “Soluciona todos los errores”
- **Respuestas generadas:** “El error ocurre porque en los archivos round_robin.c y sjf.c estás usando el tipo de dato Scheduler, pero olvidaste incluir su archivo de cabecera #include "scheduler.h" al principio del archivo. Debido a esto, el compilador no sabe qué significa Scheduler y asume que es un tipo desconocido.”
- **Errores encontrados:** Errores por modificación del tipo de dato Scheduler
- **Correcciones realizadas:** Se modificaron los archivos round_robin.c , sjf.c y fifo.c

- **Validaciones realizadas:** Se validó que los archivos no tuvieran errores al compilar

Registro 7:

- **Prompt utilizado:** “Soluciona estos errores: gcc -Wall -Wextra -std=c99 -g -o simulador_os main.c first_fit.c coalescence.c queue.c circular_queue.c stack.c fifo.c round_robin.c sjf.c”
- **Respuestas generadas:** “Los errores que te siguen apareciendo ocurren porque en los archivos round_robin.c, sjf.c y fifo.c se está redefiniendo de forma separada el tipo struct Scheduler con miembros completamente distintos. En C, no se permite tener múltiples definiciones de una misma estructura (struct Scheduler) con diferentes campos dentro de un mismo programa ejecutable, ya que el compilador y el enlazador entran en conflicto sobre qué tamaño y qué miembros tiene realmente esa estructura. Para solucionarlo conservando la limpieza de tu código, unificaremos la abstracción de los planificadores utilizando constructores especializados dentro de los módulos específicos, pero apuntando a una única interfaz común. Aquí tienes la solución exacta para cada archivo afectado”
- **Errores encontrados:** Ninguno
- **Correcciones realizadas:** Se modificaron los archivos scheduler.h y main.c
- **Validaciones realizadas:** Se validó que los archivos no tuvieran errores al compilar

Registro 8:

- **Prompt utilizado:** “En este repositorio de github: <https://github.com/Ingenieria-en-computacion-FI/proyecto-final-ketchup.git> estoy teniendo problemas al hacer la gráfica en la imagen grafica_recursos.png” ayudame a encontrar el problema.
- **Respuestas generadas:** “En tu archivo `2_graficador.py`, tienes estas dos líneas:

```
completados = df[df['Estado_Final'] == 'COMPLETADO']
bloqueados   = df[df['Estado_Final'] == 'BLOQUEADO_MEMORIA']
```

Como tus datos en el CSV traen ese espacio en blanco invisible (ej. 'COMPLETADO '), cuando Python intenta buscar la palabra exacta 'COMPLETADO', no encuentra ninguna coincidencia. Al no encontrar nada, guarda dos listas vacías y Matplotlib te dibuja un lienzo en blanco.

La solución

Tenemos que usar `.strip()` para limpiar los datos de la columna antes de filtrarlos. Aquí tienes tu archivo `2_graficador.py` con las líneas de limpieza integradas. Reemplaza tu código con este:

```
import pandas as pd

import matplotlib.pyplot as plt

def graficar_resultados(archivo_entrada = "resultados.csv"):

    try:

        df = pd.read_csv(archivo_entrada)

        # 1. Limpiamos las cabeceras (nombres de las columnas)

        df.columns = df.columns.str.strip()

        # 2. Limpiamos los datos internos de la columna para quitar espacios fantasmas

        df['Estado_Final'] = df['Estado_Final'].str.strip()

    except FileNotFoundError:

        print("Error: No se encontro el archivo. Corre el simulador en C primero.")

        return

    # Separamos los procesos por su estado final

    completados = df[df['Estado_Final'] == 'COMPLETADO']

    bloqueados = df[df['Estado_Final'] == 'BLOQUEADO_MEMORIA']

    fig, ax = plt.subplots(figsize=(10, 6))

    # Graficamos procesos completados (Azul) y bloqueados por memoria (Rojo)

    ax.bar(completados['PID'].astype(str), completados['Memoria_Usada'],

           color='steelblue', label='Completados')
```

```

if not bloqueados.empty:

    ax.bar(bloqueados['PID'].astype(str), bloqueados['Memoria_Usada'],

           color='crimson', label='Bloqueados (FALTA RAM)')

ax.set_xlabel('PID del Proceso')

ax.set_ylabel('Memoria Requerida (MB)')

ax.set_title('Uso de Memoria por Proceso y Estado de Finalizacion')

ax.legend()

ax.grid(axis='y', linestyle='--', alpha=0.7)

plt.xticks(rotation=45)

plt.tight_layout()

# Guardamos la imagen

plt.savefig("grafica_recursos.png", dpi=300)

print("Grafica guardada como 'grafica_recursos.png'.")

if __name__ == "__main__":

    graficar_resultados()

```

- **Errores encontrados:** Un espacio que estaba perjudicando todo el proceso al buscar la palabra 'COMPLETADO'
- **Correcciones realizadas:** Se hizo aplicó la función .strip() para quitar esos espacios.
- **Validaciones realizadas:** ninguna, había un error inminente.

Registro 9:

- **Prompt utilizado:** “No puedes hacer que se compile mas facilmente todo con un Make? para que no tenga que correr primer el 1_generador.py después el simulador en c, después 2_graficador.py y por último 3_analisis.py?”

- **Respuestas generadas:** Un **Makefile** no solo te ahorrará teclear en la terminal de tu Debian, sino que compilará **únicamente** los archivos que hayas modificado, ahorrando tiempo de procesamiento.

Aquí tienes el **Makefile** exacto para la arquitectura de tu simulador.

Creación del **Makefile**

Crea un archivo llamado exactamente **Makefile** (sin extensiones, con la "M" mayúscula) en la misma carpeta donde tienes todos tus archivos **.c** y **.h**, y pega este contenido.

⚠ ADVERTENCIA CRÍTICA PARA MAKEFILES: Las sangrías antes de los comandos de compilación (**\$(CC)...**, **rm -f...**, **./\$(TARGET)**) **deben ser tabulaciones reales (tecla TAB)**, no espacios en blanco. Si usas espacios, el comando **make** te arrojará un error.

```
# =====

# UNAM - Makefile para Simulador de SO

# =====

# Variables del compilador

CC = gcc

# Banderas: -Wall y -Wextra muestran advertencias
útiles, -g permite usar depuradores como GDB

CFLAGS = -Wall -Wextra -g

# Nombre del ejecutable final

TARGET = simulador

# Lista de archivos fuente (.c)

SRCS = main.c process.c memory_manager.c scheduler.c
circular_queue.c queue.c stack.c coalescence.c
```



```
# Convierte dinámicamente la lista de .c en una lista
de archivos objeto (.o)
```

```
OBJS = $(SRCS:.c=.o)
```

```
# Regla por defecto que se ejecuta al escribir solo
'make'
```

```
all: $(TARGET)
```

```
# Regla para enlazar los archivos objeto y crear el
ejecutable
```

```
$(TARGET): $(OBJS)
```

```
    $(CC) $(CFLAGS) -o $(TARGET) $(OBJS)
```

```
# Regla genérica para compilar cada .c en su
respectivo .o
```

```
# $< es el primer prerrequisito (el .c) y $@ es el
nombre del objetivo (el .o)
```

```
%.o: %.c
```

```
    $(CC) $(CFLAGS) -c $< -o $@
```

```
# Regla para limpiar los binarios y compilar desde
cero
```

```
clean:
```

```
    rm -f $(OBJS) $(TARGET)
```

```
# Regla personalizada para compilar (si es necesario)
y ejecutar automáticamente
```

```
run: $(TARGET)
```

```
./$(TARGET)
```

A partir de ahora, tu flujo de ejecución completo (C + Python) se ve así:

```
python3 1_generador.py
```

```
make run
```

```
python3 2_graficador.py
```

```
python3 3_analisis.py
```

- **Errores encontrados:** Ninguno.
- **Correcciones realizadas:** Se creó el makefile.
- **Validaciones realizadas:** Ninguna.

Registro 10:

- **Prompt utilizado:** “Según estos cambios en el repositorio: <https://github.com/Ingenieria-en-computacion-FI/proyecto-final-ketchup.git> como quedaría un nuevo make file para compilar todo esto”
- **Respuestas generadas:** “Aquí tienes el nuevo archivo **Makefile** para tu proyecto.

Esta versión soluciona el problema de las múltiples definiciones de **main** (filtrando **src/main.c** al compilar las pruebas), incluye la bandera **-g** por si necesitas meterle GDB o Valgrind desde tu terminal para rastrear fugas de memoria, y escanea recursivamente todos los subdirectorios nuevos que acabamos de crear.

```
CC=gcc
```

```
CFLAGS=-Wall -Wextra -std=c11 -Iinclude -g
```

```
# Definimos explícitamente todos los directorios de tu  
nueva arquitectura
```

```
SRC_DIRS = src src/algorithms src/memory src/scheduler  
src/utils
```

```
# Extraemos todos los archivos .c de esos directorios

SRC = $(foreach dir, $(SRC_DIRS), $(wildcard $(dir)/*.c))

# Extraemos los archivos de pruebas

TESTS = $(wildcard tests/*.c)

# Regla por defecto: limpia primero y luego compila

all: clean build

# Compilación del núcleo del simulador

build:

    mkdir -p bin

    $(CC) $(CFLAGS) $(SRC) -o bin/main

# Compila y ejecuta automáticamente

run: build

    ./bin/main

# Compilación de pruebas de integración

# Se usa filter-out para excluir src/main.c y que no
choque con tests/main.c

test: clean

    mkdir -p bin

    $(CC) $(CFLAGS) $(TESTS) $(filter-out src/main.c,
$(SRC)) -o bin/tests
```

```

./bin/tests

# Limpieza de binarios

clean:

    rm -rf bin/*

```

⚠ Un detalle vital al copiar y pegar:

Recuerda que GNU Make es extremadamente estricto con la indentación. Los espacios en blanco antes de los comandos (`mkdir`, `$(CC)`, `rm`) **deben ser Tabulaciones (Tab ↵)**, nunca barras espaciadoras. Si usas Vim o algún otro editor en consola para pegarlo, asegúrate de que no te haya convertido los tabs en espacios automáticamente, o te lanzará el clásico error de *missing separator*.

- **Errores encontrados:** Ninguno.
- **Correcciones realizadas:** Se editó nuevamente todo el makefile
- **Validaciones realizadas:** Ninguna.

Registro 11:

- **Prompt utilizado:** “me salto este output al hacer make run, es normal?
src/algorithms/brute_force.c: In function 'algo_brute_force_find':
src/algorithms/brute_force.c:9:9: warning: variable 'holes_found' set but not used [-Wunused-but-set-variable=]
9 | int holes_found = 0;
| ^~~~~~
/usr/bin/ld: /tmp/ccjQDWv9.o: in function `scheduler_add_process':
/home/danieln/Documentos/Lenguaje
C/EDA/proyecto-final-ketchup/src/scheduler/round_robin.c:33: multiple
definition of `scheduler_add_process';
/tmp/cchjMzBw.o:/home/danieln/Documentos/Lenguaje
aje C/EDA/proyecto-final-ketchup/src/scheduler/fifo.c:24: first defined here
/usr/bin/ld: /tmp/ccjQDWv9.o: in function `scheduler_next':
/home/danieln/Documentos/Lenguaje
C/EDA/proyecto-final-ketchup/src/scheduler/round_robin.c:44: multiple
definition of `scheduler_next';
/tmp/cchjMzBw.o:/home/danieln/Documentos/Lenguaje C/E

DA/proyecto-final-ketchup/src/scheduler/fifo.c:32: first defined here
 /usr/bin/ld: /tmp/ccjQDWv9.o: in function `scheduler_is_done':
 /home/danieln/Documentos/Lenguaje
 C/EDA/proyecto-final-ketchup/src/scheduler/round_robin.c:76: multiple
 definition of `scheduler_is_done';
 /tmp/cchjMzBw.o:/home/danieln/Documentos/Lenguaje
 C/EDA/proyecto-final-ketchup/src/scheduler/fifo.c:39: first defined here
 /usr/bin/ld: /tmp/ccjQDWv9.o: in function `scheduler_destroy':
 /home/danieln/Documentos/Lenguaje
 C/EDA/proyecto-final-ketchup/src/scheduler/round_robin.c:83: multiple
 definition of `scheduler_destroy';
 /tmp/cchjMzBw.o:/home/danieln/Documentos/Lenguaje
 C/EDA/proyecto-final-ketchup/src/scheduler/fifo.c:46: first defined here
 /usr/bin/ld: /tmp/ccwLt71m.o: in function `scheduler_create_fifo':
 /home/danieln/Documentos/Lenguaje
 C/EDA/proyecto-final-ketchup/src/scheduler/scheduler.c:29: multiple definition
 of `scheduler_create_fifo';
 /tmp/cchjMzBw.o:/home/danieln/Documentos/Lenguaje
 e C/EDA/proyecto-final-ketchup/src/scheduler/fifo.c:13: first defined here
 /usr/bin/ld: /tmp/ccwLt71m.o: in function `scheduler_create_rr':
 /home/danieln/Documentos/Lenguaje
 C/EDA/proyecto-final-ketchup/src/scheduler/scheduler.c:39: multiple definition
 of `scheduler_create_rr';
 /tmp/ccjQDWv9.o:/home/danieln/Documentos/Lenguaje
 C/EDA/proyecto-final-ketchup/src/scheduler/round_robin.c:22: first defined
 here
 /usr/bin/ld: /tmp/ccwLt71m.o: in function `scheduler_add_process':
 /home/danieln/Documentos/Lenguaje
 C/EDA/proyecto-final-ketchup/src/scheduler/scheduler.c:60: multiple definition
 of `scheduler_add_process';
 /tmp/cchjMzBw.o:/home/danieln/Documentos/Lenguaje
 e C/EDA/proyecto-final-ketchup/src/scheduler/fifo.c:24: first defined here
 /usr/bin/ld: /tmp/ccwLt71m.o: in function `scheduler_next':
 /home/danieln/Documentos/Lenguaje
 C/EDA/proyecto-final-ketchup/src/scheduler/scheduler.c:91: multiple definition
 of `scheduler_next'; /tmp/cchjMzBw.o:/home/danieln/Documentos/Lenguaje
 C/EDA
 /proyecto-final-ketchup/src/scheduler/fifo.c:32: first defined her
 /usr/bin/ld: /tmp/ccwLt71m.o: in function `scheduler_is_done':
 /home/danieln/Documentos/Lenguaje
 C/EDA/proyecto-final-ketchup/src/scheduler/scheduler.c:138: multiple
 definition of `scheduler_is_done';
 /tmp/cchjMzBw.o:/home/danieln/Documentos/Lenguaje C
 /EDA/proyecto-final-ketchup/src/scheduler/fifo.c:39: first defined here

```

/usr/bin/ld: /tmp/ccwLt71m.o: in function `scheduler_destroy':
/home/danieln/Documentos/Lenguaje
C/EDA/proyecto-final-ketchup/src/scheduler/scheduler.c:147: multiple
definition of `scheduler_destroy';
/tmp/cchjMzBw.o:/home/danieln/Documentos/Lenguaje C
/EDA/proyecto-final-ketchup/src/scheduler/fifo.c:46: first defined here
/usr/bin/ld: /tmp/cc7D336y.o: in function `scheduler_create_sjf':
/home/danieln/Documentos/Lenguaje
C/EDA/proyecto-final-ketchup/src/scheduler/sjf.c:13: multiple definition of
`scheduler_create_sjf';
/tmp/ccwLt71m.o:/home/danieln/Documentos/Lenguaje C/EDA
/proyecto-final-ketchup/src/scheduler/scheduler.c:50: first defined here
/usr/bin/ld: /tmp/cc7D336y.o: in function `scheduler_add_process':
/home/danieln/Documentos/Lenguaje
C/EDA/proyecto-final-ketchup/src/scheduler/sjf.c:22: multiple definition of
`scheduler_add_process';
/tmp/cchjMzBw.o:/home/danieln/Documentos/Lenguaje C/ED
A/proyecto-final-ketchup/src/scheduler/fifo.c:24: first defined here
/usr/bin/ld: /tmp/cc7D336y.o: in function `scheduler_next':
/home/danieln/Documentos/Lenguaje
C/EDA/proyecto-final-ketchup/src/scheduler/sjf.c:44: multiple definition of
`scheduler_next'; /tmp/cchjMzBw.o:/home/danieln/Documentos/Lenguaje
C/EDA/proye
cto-final-ketchup/src/scheduler/fifo.c:32: first defined here
/usr/bin/ld: /tmp/cc7D336y.o: in function `scheduler_is_done':
/home/danieln/Documentos/Lenguaje
C/EDA/proyecto-final-ketchup/src/scheduler/sjf.c:59: multiple definition of
`scheduler_is_done'; /tmp/cchjMzBw.o:/home/danieln/Documentos/Lenguaje
C/EDA/pr
oyecto-final-ketchup/src/scheduler/fifo.c:39: first defined here
/usr/bin/ld: /tmp/cc7D336y.o: in function `scheduler_destroy':
/home/danieln/Documentos/Lenguaje
C/EDA/proyecto-final-ketchup/src/scheduler/sjf.c:66: multiple definition of
`scheduler_destroy'; /tmp/cchjMzBw.o:/home/danieln/Documentos/Lenguaje
C/EDA/pr
oyecto-final-ketchup/src/scheduler/fifo.c:46: first defined here
collect2: error: ld returned 1 exit status
make: *** [Makefile:14: build] Error 1

```

- **Respuestas generadas:**

“Sí, este *output* es completamente normal cuando se reestructura un proyecto en C. El compilador (y específicamente el enlazador o "linker", **ld**) está

haciendo su trabajo al detectar un par de colisiones en la arquitectura de los archivos.

Aquí te explico exactamente qué significan estos mensajes y cómo solucionarlos en dos pasos sencillos.

1. El error crítico: Múltiples Definiciones (**multiple definition of...**)

El error fatal que detiene la compilación (**ld returned 1 exit status**) ocurre porque el enlazador encontró la misma función declarada varias veces en distintos archivos.

¿Por qué pasa esto? Tu nuevo **Makefile** está compilando **todos** los archivos **.c** dentro de la carpeta **src/scheduler/**. Sin embargo, si revisas esa carpeta, tienes dos enfoques conviviendo al mismo tiempo:

1. Archivos individuales por política: **fifo.c**, **round_robin.c**, **sjf.c**.
2. Un archivo unificado: **scheduler.c** (el cual incluye la lógica de los tres algoritmos usando un **switch** o **if/else** interno y un atributo **type**).

Como todos estos archivos intentan crear funciones con el mismo nombre exacto (como **scheduler_add_process** o **scheduler_next**), el compilador choca porque no sabe cuál de todas las versiones debe inyectar en el binario final.

La solución: Dado que **scheduler.c** es un archivo maestro que unifica todas las políticas (FIFO, RR, SJF) y actúa como un despachador global para tu simulador, los archivos individuales ya son obsoletos.

Debes eliminar (o mover fuera de la carpeta **src/**) los archivos redundantes. Desde tu terminal, ejecuta:

```
rm src/scheduler/fifo.c
rm src/scheduler/round_robin.c
rm src/scheduler/sjf.c
```

(Dejando únicamente **src/scheduler/scheduler.c** dentro de esa carpeta).

2. El Warning: Variable declarada pero no usada (**-Wunused-but-set-variable**)

El primer mensaje que ves no es un error, sino una advertencia de limpieza de código (**warning**).

En el archivo `src/algorithms/brute_force.c`, en la línea 9, declaramos una variable `int holes_found = 0;` y más abajo se incrementa (`holes_found++;`). Sin embargo, en la versión final de ese algoritmo no estamos retornando ni imprimiendo (con `printf` o `log_event`) esa variable. Como tienes activada la bandera `-Wall -Wextra` en tu compilador, GCC te advierte que estás gastando un espacio de memoria para algo que no se usa.

La solución: Abre `src/algorithms/brute_force.c` y simplemente borra la línea 9 y la línea donde se incrementa. El código debe quedar más limpio, así:

```
#include <stdio.h>

#include "algorithms.h"

MemoryBlock* algo_brute_force_find(MemoryManager* mm, int size) {

    if (!mm || !mm->head) return NULL;

    MemoryBlock* current = mm->head;

    MemoryBlock* valid_candidate = NULL;

    // Exploración exhaustiva
    while (current != NULL) {

        if (current->free) {

            if (current->size >= size && valid_candidate == NULL) {

                valid_candidate = current;

            }

        }

        current = current->next;

    }

    return valid_candidate;

}
```


3. Recompilar

Una vez que hayas eliminado los 3 archivos repetidos del scheduler y limpiado la variable de fuerza bruta, ejecuta en tu terminal:

```
make clean
```

```
make run
```

Al hacer el **make clean** eliminas los archivos objeto corruptos del intento anterior (.o), y con el nuevo **make run** la compilación pasará limpia y tu simulador se ejecutará correctamente.

- **Errores encontrados:** Choque de declaraciones de funciones en varios archivos y una variable sin usar.
- **Correcciones realizadas:** Eliminación de los archivos src/scheduler/fifo.c src/scheduler/round_robin.c y src/scheduler/sjf.c y variable "holes_found" en src/algorithms/brute_force.c
- **Validaciones realizadas:** Posibles errores en toda la estructura del código.

Registro 12:

- **Prompt utilizado:** "No podría hacer un make que también compile los archivos en python?"

La simulación en general viene dada por:

- Generación de procesos (que los genera el archivo ../scripts/1_generador.py) en un archivo .csv
- Simulación de procesos (todo el simulador en c) que toma los procesos de "Procesos.csv"
- Graficación de procesos (que lo hace el archivo ../scripts/2_graficador.py)
- Análisis de resultados (que lo hace el archivo ../scripts/3_analisis.py)

O dime otras propuestas para mejorar la eficiencia de estos procesos”

- **Respuesta generada:** “¡Totalmente! De hecho, `make` fue diseñado originalmente en los sistemas UNIX para automatizar exactamente este tipo de *pipelines* o flujos de trabajo en cadena, no solamente para compilar código C.

Aquí tienes la forma de integrarlo directamente en tu **Makefile**, junto con otras propuestas arquitectónicas para mejorar la eficiencia del simulador.

1. La solución directa: Actualizar el Makefile

Puedes agregar una nueva regla (por ejemplo, `simulate`) al final de tu `Makefile` actual. Esta regla compilará el código (si es necesario) y luego ejecutará los cuatro pasos en orden temporal.

Agrega esto al final de tu archivo `Makefile`:

```
# ... (tu código anterior del Makefile) ...

# Regla para ejecutar el flujo completo de simulación
simulate: build

    @echo "\n=== 1. GENERANDO CARGA DE PROCESOS ==="

    python3 scripts/1_generador.py

    @echo "\n=== 2. EJECUTANDO KERNEL DEL SIMULADOR ==="

    ./bin/main

    @echo "\n=== 3. GENERANDO GRÁFICAS DE MEMORIA ==="

    python3 scripts/2_graficador.py

    @echo "\n=== 4. ANALIZANDO TENDENCIAS DE CPU ==="

    python3 scripts/3_analisis.py

    @echo "\n=== SIMULACIÓN FINALIZADA CON ÉXITO ==="
```

(Nota: Recuerda usar Tabulaciones `Tab` ↵ para los espacios en blanco debajo de `simulate:`).

Para ejecutar todo de un solo golpe, ahora solo tienes que escribir en tu terminal de Linux:

```
make simulate
```

Propuestas para mejorar la eficiencia del proceso

Aunque la regla `simulate` de arriba funciona perfecto, en entornos de desarrollo Linux orientados a sistemas y automatización, existen enfoques más eficientes dependiendo de lo que quieras lograr. Aquí tienes tres propuestas:

Propuesta A: Dependencias reales en Make (Nivel Avanzado)

El problema de la regla `simulate` anterior es que **siempre** va a correr todos los pasos, incluso si los datos no han cambiado. La filosofía real de `make` es rastrear las fechas de modificación de los archivos.

Puedes estructurar tu Makefile para que sepa qué archivo genera a cuál. Así, si solo borras `grafica_recursos.png`, `make` sabrá que solo necesita correr el script 2, sin tener que volver a simular todo el SO.

```
# Generar CSV solo si el script cambia o no existe el CSV
```

```
Procesos.csv: scripts/1_generador.py
```

```
    python3 scripts/1_generador.py
```

```
# Correr el simulador solo si el binario o los procesos de
entrada cambian
```

```
resultados.csv: bin/main Procesos.csv
```

```
    ./bin/main
```

```
# Graficar solo si hay nuevos resultados de simulación
```

```
grafica_recursos.png: scripts/2_graficador.py resultados.csv
```

```
    python3 scripts/2_graficador.py
```

```
# Unificar todo
```

```
flujo_inteligente: grafica_recursos.png
```

```
python3 scripts/3_analisis.py
```

Propuesta B: Invocación interna desde el Kernel en C (System Calls)

En lugar de depender de un ente externo (como Make) para orquestar la simulación, puedes hacer que tu sistema operativo sea "autoconsciente" y llame a sus propios periféricos (los scripts).

En tu `src/main.c`, puedes usar la librería estándar `<stdlib.h>` para ejecutar comandos de terminal directamente desde el binario en C, justo antes de leer los archivos y justo después de cerrarlos:

```
int main() {

    log_event("Iniciando Simulador...\n");

    // 1. C llama a Python para generar el hardware virtual

    log_event("Generando procesos mediante sub-rutina...\n");

    system("python3 scripts/1_generador.py");

    // ... (Tu código de FASE 1 leyendo Procesos.csv) ...

    // ... (Tu código de FASE 4 escribiendo resultados.csv)
    ...

    // 2. C llama a Python para graficar justo antes de hacer
    el return 0

    log_event("Levantando interfaz gráfica y análisis
estadístico...\n");

    system("python3 scripts/2_graficador.py");

    system("python3 scripts/3_analisis.py");

    return 0;
```

```
}
```

Esto es muy eficiente porque encapsula toda la lógica. Simplemente ejecutas `./bin/main` y el binario en C se encarga de despertar a Python cuando lo necesita.

Propuesta C: Un Orquestador de Benchmarking en Python (`subprocess`)

Si en un futuro quieres evaluar qué algoritmo es mejor (Best Fit vs Worst Fit, o FIFO vs Round Robin), vas a necesitar correr el simulador 100 o 500 veces seguidas, cambiar parámetros, sacar promedios y graficar el rendimiento. Hacer un bucle de 500 iteraciones en Make es incómodo.

Puedes crear un archivo `scripts/benchmark.py` que utilice la librería `subprocess` para invocar al binario de C iterativamente:

```
import subprocess

import os


def correr_benchmarking(iteraciones=100):

    # Compilar el proyecto en C automáticamente

    subprocess.run(["make", "build"])


    for i in range(iteraciones):

        print(f"--- Iteración {i+1} ---")

        # Generar procesos

        subprocess.run(["python3", "scripts/1_generador.py"])


        # Ejecutar el binario de C

        subprocess.run(["./bin/main"])
```

```
# Opcional: Renombrar resultados.csv a
resultados_1.csv, resultados_2.csv, etc.
```

```
os.rename("resultados.csv",
f"data/benchmarks/resultados_{i}.csv")
```

```
print("Benchmarking completado. Listo para graficar
promedios masivos.")
```

Conclusión: Si solo quieres presionar un botón y ver que todo funcione, **modificar el Makefile con la regla `simulate`** es la vía más rápida y natural ahora mismo. Si planeas hacer pruebas de estrés rigurosas a tus algoritmos de memoria en el futuro, el **orquestador en Python** será tu mejor aliado.

- **Errores encontrados:** Ninguno.
- **Correcciones realizadas:** Ninguna, solamente generación de ideas de como cambiar el make.
- **Validaciones realizadas:** Ninguna.

Registro 13:

- **Prompt utilizado:** “me gusta la propuesta de simulate, pero no se podría buscar directamente el archivo .csv que genera 1_generador.py en ../scripts para el simulador en c y los demas en archivos en ../scripts para que busquen el archivo resultados.csv que genera main.c?”
- **Respuesta generada:** “¡Sí, absolutamente! De hecho, hacer eso es lo ideal para tener un proyecto ordenado y profesional.

El comportamiento actual ocurre por cómo manejan los sistemas operativos el **Directorio de Trabajo Actual (Current Working Directory)**. Cuando ejecutas `make simulate` desde la raíz del proyecto, todos los scripts de

Python y el binario de C heredan la raíz como su directorio de trabajo. Por eso, al usar nombres de archivos secos como "**Procesos.csv**" o "**resultados.csv**", todo se genera y se busca en la raíz, llenando tu carpeta principal de archivos temporales.

Aprovechando la excelente estructura de carpetas que tienes definida en tu **README.md**, podemos modificar los paths internos de tus archivos para que **1_generador.py** guarde directamente en **data/inputs/**, el simulador en C lo busque ahí, genere sus resultados en **data/outputs/**, y los scripts de graficación y análisis lean directamente de esa carpeta de salida.

Aquí tienes los cambios exactos que debes realizar en cada archivo para lograr este flujo limpio y directo:

1. Modificación en **scripts/1_generador.py**

Cambiamos la ruta por defecto para que los procesos simulados se guarden en la carpeta de entradas:

```
# Modifica la línea de la definición de la función (línea 4 aprox.)

def                                generador_procesos(cantidad=25,
archivo_salida='data/inputs/Procesos.csv'):

    # El resto del código se queda exactamente igual...
```

2. Modificación en **src/main.c**

Actualizamos los descriptores de archivos para que el Kernel lea de la carpeta de entradas y escriba sus métricas en la carpeta de salidas:

```
// FASE 1: Cambia la ruta de lectura del CSV (línea 31 aprox.)

FILE* fp_in = fopen("data/inputs/Procesos.csv", "r");


// FASE 4: Cambia la ruta de exportación del reporte (línea 73 aprox.)

FILE* fp_out = fopen("data/outputs/resultados.csv", "w");
```

3. Modificación en **scripts/2_graficador.py**

Actualizamos el script para que busque el archivo intermedio en la carpeta de salidas y guarde la imagen final en la carpeta de reportes visuales:

```
# Modifica la ruta de entrada por defecto (línea 4
aprox.)

def graficar_resultados(archivo_entrada =
"data/outputs/resultados.csv"):

    # ... código de lectura ...

    # Modifica la línea final donde guardas la imagen
    (línea 40 aprox.)

    plt.savefig("reports/png/grafica_recursos.png",
dpi=300)

    print("Grafica guardada como
'reports/png/grafica_recursos.png'.")
```

4. Modificación en `scripts/3_analisis.py` Hacemos lo mismo con el script de análisis estadístico y dispersión:

```
# Modifica la ruta de entrada por defecto (línea 5 aprox.)
def
analizar_tendencias(archivo_entrada="data/outputs/resultados.
csv"):
# ... código de análisis ...

    # Modifica la línea final donde guardas la
    dispersión (línea 31 aprox.)
plt.savefig("reports/png/analisis_dispersion.png", dpi=300)
    print("Análisis estadístico guardado como
'reports/png/analisis_dispersion.png'.")
```

¿Cómo queda tu **Makefile** ahora?

Tu **Makefile** no necesita cambiar los comandos de ejecución, pero ahora puedes estar seguro de que la raíz de tu repositorio se mantendrá completamente limpia de archivos `.csv` o `.png` sueltos. La regla queda impecable:


```
CC=gcc
```

```
CFLAGS=-Wall -Wextra -std=c11 -Iinclude -g
```

```
SRC_DIRS = src src/algorithms src/memory src/scheduler src/utils
```

```
SRC = $(foreach dir, $(SRC_DIRS), $(wildcard $(dir)/*.c))
```

```
all: clean build
```

```
build:
```

```
    mkdir -p bin
```

```
    $(CC) $(CFLAGS) $(SRC) -o bin/main
```

```
simulate: build
```

```
    @echo "\n=== 1. GENERANDO CARGA EN DATA/INPUTS ==="
```

```
    python3 scripts/1_generador.py
```

```
    @echo "\n=== 2. EJECUTANDO KERNEL DEL SIMULADOR ==="
```

```
    ./bin/main
```

```
    @echo "\n=== 3. GENERANDO GRÁFICAS EN REPORTS/PNG ==="
```

```
    python3 scripts/2_graficador.py
```

```
    @echo "\n=== 4. ANALIZANDO TENDENCIAS DE CPU ==="
```

```
    python3 scripts/3_analisis.py
```

```
    @echo "\n=== SIMULACIÓN EN CADENA CONCLUIDA ==="
```

```
clean:
```

```
    rm -rf bin/*
```

Al ejecutar `make simulate`, el flujo de datos será totalmente transparente y estructurado:

1. Python escribe en `data/inputs/`.
2. C lee de `data/inputs/` y escribe en `data/outputs/`.
3. Los scripts de análisis leen de `data/outputs/` y guardan los reportes finales limpios en `reports/png/`.

Esto le da un nivel de madurez de software real a tu proyecto integrador.

- **Errores encontrados:** Ninguno.
- **Correcciones realizadas:** Modificaciones en `main.c` y los archivos en python
- **Validaciones realizadas:** Ninguna.

Registro 14:

- **Prompt utilizado:** “Es normal que la gráfica se comporte de esta manera?”
- **Respuesta generada:** “¡Qué excelente noticia que el *pipeline* automatizado ya haya funcionado! Ver la gráfica generada con los datos reales es un gran paso.

Yendo a tu pregunta: **sí, es completamente normal que la gráfica se comporte así**, y de hecho, está ilustrando a la perfección un fenómeno clásico en el diseño de sistemas operativos. La gráfica no está mal, simplemente está revelando una limitación en la arquitectura de tu Kernel actual.

¿Qué está pasando realmente en la RAM?

Si tomas una calculadora y sumas la memoria de los primeros procesos azules (completados), verás que el resultado ronda exactamente los **1024 MB**.

El problema radica en cómo está programado el ciclo de vida de los procesos en tu `src/main.c`. Actualmente, tu simulador opera como un sistema de **cargas por lotes estáticos**, no de forma dinámica.

Esto es lo que ocurre cronológicamente:

1. **El Cuello de Botella Estático (Fase 2):** Tu Kernel intenta cargar absolutamente todos los procesos a la RAM *al mismo tiempo*, antes de que el procesador empiece a trabajar. Cuando la memoria alcanza su límite de 1024 MB (aproximadamente en el noveno proceso), todos los demás procesos en la fila son rechazados automáticamente y etiquetados como **BLOCKED**.
2. **Ejecución Ciega (Fase 3):** Después de esa asignación inicial masiva, arranca tu procesador simulado (Round Robin). La CPU hace su trabajo perfecto: ejecuta los procesos azules, termina sus ráfagas, libera su espacio de memoria con `mm_free()` y lo unifica con `mm_coalesce()`.
3. **El Defecto:** Aunque la memoria se queda vacía conforme los procesos azules terminan, tu simulador *nunca voltea hacia atrás* para revisar a los procesos rojos que se quedaron en estado **BLOCKED** para ingresarlos a la memoria recién liberada. Se quedaron bloqueados para siempre en la Fase 2.

La Solución Arquitectónica

En una clase de Sistemas Operativos a nivel de ingeniería, esto se resume en que lograste implementar exitosamente el **Planificador a Corto Plazo** (el despachador de la CPU), pero te hace falta la lógica del **Planificador a Largo Plazo** (el despachador de trabajos).

Para que la gráfica termine completamente azul (es decir, que la simulación logre correr los 25 procesos), la validación de memoria no debe ocurrir de golpe al principio. Debe ocurrir dinámicamente *dentro* del mismo ciclo `while` de la CPU: si hay espacio en RAM, metes un proceso de la cola de espera; si se llena, ejecutas la CPU para que alguien termine y haga espacio.

¿Te gustaría que reestructuremos el ciclo principal en tu `main.c` para darle este comportamiento dinámico al administrador de memoria y lograr que todos los procesos se completen?”

- **Errores encontrados:** Ninguno.
- **Correcciones realizadas:** Ninguna.
- **Validaciones encontradas:** Que el sistema se encuentra bien y los resultados son correctos.

